Distributed and Parallel Computing
OpenMP Report

Pedro Rio
MSc Data Science and Engineering
Instituto Superior Técnico
97241

Sebastião Maia Serqueira
MSc Data Science and Engineering
Instituto Superior Técnico
97108

Eduardo Machado
MSc Data Science and Engineering
Instituto Superior Técnico
97133

1 Program Overview

The program serial implementation is divided into four main parts:

1.1 ReadInput

A for loop is used to read each line of the instance file and get the number of iterations, users, items, latent features, non zero elements and the value of the convergence coefficient. After this step, the matrix A and a vector of the indexes of the non zero elements are then populated.

1.2 InitialLR

The routine provided was used to initialise the matrices L and R.

1.3 UpdateLR

A recommendation score is predicted for each one of the non zero elements of matrix A, using its i and j indexes to compute the internal product of the row i of the matrix L and the column j of matrix R. Afterwards, the differential between the real value of the non zero element in matrix A and its prediction is computed for all the non zero elements. Finally and also for each one of the non zero elements of matrix A, the selected column of matrix L and a row of R are updated to decrease the obtained differential as a function of the existing differential, the convergence coefficient and the stored matrices L and R. The function is repeated in a loop for the number of iterations and for each iteration the program stores the values of the matrices L and R and uses them to update the matrices L and R.

1.4 FilterFinalMatrix

The matrix B is computed as the multiplication of matrices L and R. Afterwards, for each row of matrix B, the maximum value is computed without taking into account the the non zero elements. The location of the values are stored in a vector and streamed to the standard output.

2 OpenMP Version

2.1 Decomposition

The factorisation is a data-parallel computation. `updateLR` divides the non-zero list across threads — each computes predictions and gradient increments for its share — and the final stage divides matrix B by rows. `initialLR` is left sequential: it draws pseudo-random numbers whose order is part of the specification, so parallelising it would race on the generator and change the result.

2.2 One parallel region for the whole run

Rather than open a parallel region inside every iteration — which forks and joins a thread team *numberOfIterations* times — we open *one* region in `matFact` that spans the entire iteration loop. The worksharing constructs inside `updateLR` are *orphaned*: they bind to that enclosing region, so the team is created once and reused for every iteration.

2.3 Synchronization

Each iteration must read a consistent snapshot of L and R. The two snapshot copies are independent of each other, so they carry `nowait`; the prediction loop keeps its implicit barrier, which orders the snapshot writes before the update reads them. The barrier at the end of the last worksharing loop preserves the sequential dependence between iterations, so no thread races ahead into the next one.

3 Optimisations

Every block except `initialLR` is parallelised. The interesting work was in *how* the gradient update is parallelised, because that block carries the only real data hazard.

3.1 Lock-free update via array reduction

The update is the hot block. Two non-zeros that share a user write the same row of L, and two that share an item write the same column of R, so a naive parallel loop has write conflicts. Guarding each write with `atomic` would serialise exactly the inner loop that should run in parallel. Instead we use an array reduction, `reduction(+ : L[:lSize], R[:rSize])`: every thread accumulates into a private, zero-initialised copy of L and R — the increments read only the snapshots `StoreL`/`StoreR`, never the live factors — and the runtime sums the private copies into L and R at the barrier. No atomics, no lock contention. Replacing the atomics with the reduction was the change that moved the program from *barely matching* serial to a real speedup.

3.2 Conflict-free element loops

Where the output partition is naturally disjoint, no synchronisation is needed at all. In `computeB` each thread owns a block of rows i, so no two threads ever write the same $B[i][j]$; the loop also keeps the cache-friendly i, k, j order so each thread streams R and B with unit stride. The non-zero mask and the per-user arg-max in `filterFinalMatrix` are partitioned by row in the same way.

3.3 Scheduling and output

The element loops use `schedule(static)` (uniform work per row); the per-user arg-max uses `schedule(guided)`, since rows with more items cost more. The single `std::cout` loop runs after the region closes, so output stays ordered without an `ordered` clause.

4 Implications for Parallel and Distributed Computing

4.1 Persistent threads beat repeated fork/join

Hoisting the parallel region trades many team setups for one. With short iterations the fork/join cost is not negligible relative to the work, so paying it once — enabled by orphaned worksharing — removes overhead that no amount of in-loop tuning could. The lesson generalises: keep the parallel team alive across the outer loop and share work inside it.

4.2 Resolving write conflicts: a memory-for-contention trade

The same hazard — many updates landing on the same L/R entries — admits two answers: atomics (serialise the conflicting writes) or a reduction (give each thread a private copy and combine once). The reduction costs extra memory, one copy of the factors per thread, but removes *all* contention from the inner loop. On this kernel that trade is decisively worth it, and it is the same trade that reappears in the distributed version as a gradient `Allreduce`.

4.3 Bandwidth, not cores, sets the ceiling

The kernel streams L and R far more than it computes, so it is memory-bound. The speedup therefore flattens once a socket's memory bandwidth saturates — around four threads on our machine — rather than at the core count. The serial i, k, j cache order is what raises that ceiling; an algorithm that is cache-inefficient in serial cannot be rescued by threads.

4.4 Synchronise exactly as much as the dependencies require

`nowait` on the independent snapshot copies, a barrier only at the true snapshot-then-read dependence, and the implicit end-of-loop barrier to order the sequential iterations: over-synchronising wastes the threads that finish early, while under-synchronising is a data race. Reading the dependency structure off the serial code is what tells you where each barrier must — and must not — go.

4.5 Amdahl in practice

Sequential `initialLR` and the reduction's combine step (whose cost grows with the thread count) are the non-parallel fractions, so by Amdahl's law the speedup is sublinear and eventually plateaus — consistent with the measurements below.

5 Evaluation

Figure 1: Speedup

Table 1: Speedup — **course cluster**, original submission. Not regenerable from this repository: five of these instances are not in `instances/`. NaN marks instances whose serial time rounded to zero.

fileName	1 Thread	2 Threads	4 Threads	8 Threads
inst0.in	NaN	NaN	NaN	NaN
inst1.in	NaN	NaN	NaN	NaN
inst1000-1000-100-2-30.in	0.9628647	1.753623	3.270270	4.172414
inst2.in	NaN	NaN	NaN	NaN
inst200-10000-50-100-300.in	1.0146163	1.880361	3.470833	6.170370
inst30-40-10-2-10.in	0.7500000	1.500000	1.500000	1.500000
inst400-50000-30-200-500.in	0.7269504	1.567278	2.953890	4.361702
inst500-500-20-2-100.in	0.8542435	1.618881	2.676301	2.858025
inst50000-5000-100-2-5.in	0.8337190	1.446101	2.703108	3.283854
inst600-10000-10-40-400.in	0.8638906	1.497845	2.657744	2.982833
instML100k.in	0.7516242	1.401677	2.588640	2.990060
instML1M.in	0.4093360	1.221217	3.304762	4.180723

In our opinion, our OpenMP implementation improves the speedup when it has more threads working as supposed. Nevertheless, we know by Amdahl's law that the speed is limited by the total time needed for the execution of the sequential part. As our program is not completely parallelized, the speedup cannot increase linearly with the amount of threads.

Figure 2: Total Execution Time

Table 2: Total execution time — **course cluster**, original submission. Not regenerable from this repository.

fileName	1 Thread	2 Threads	4 Threads	8 Threads
inst0.in	0	0	0	0
inst1.in	2	2	0	2
inst1000-1000-100-2-30.in	377	207	111	87
inst2.in	0	2	2	2
inst200-10000-50-100-300.in	821	443	240	135
inst30-40-10-2-10.in	8	4	4	4
inst400-50000-30-200-500.in	1410	654	347	235
inst500-500-20-2-100.in	1084	572	346	324
inst50000-5000-100-2-5.in	3025	1744	933	768
inst600-10000-10-40-400.in	1609	928	523	466
instML100k.in	2001	1073	581	503
instML1M.in	5934	1989	735	581

5.1 Measured again on this machine

The tables above are the original cluster submission, kept for the large-instance evidence they carry. The tables below were measured by `compare/scripts/benchmark.py` on an Apple Silicon laptop (macOS 15.6, 10 cores, 8 of them performance cores), three repetitions with the minimum kept, every run verified against the instance's `.out` file; every figure traces to a row in `compare/data/comparison.omp.csv`.

The two machines are not comparable, and no figure below is divided by a figure above.

Table 3: Total execution time, seconds — **this laptop**, regenerable from this repository.

fileName	1 Thread	2 Threads	4 Threads	8 Threads
inst1000-1000-100-2-30.in	4.77	2.80	1.74	1.95
inst200-10000-50-100-300.in	7.57	5.19	3.84	4.32
inst500-500-20-2-100.in	10.39	6.11	4.32	4.33

Table 4: Speedup over the serial baseline measured in the same run — **this laptop**.

fileName	1 Thread	2 Threads	4 Threads	8 Threads
inst1000-1000-100-2-30.in	0.94	1.60	2.57	2.30
inst200-10000-50-100-300.in	0.88	1.29	1.74	1.55
inst500-500-20-2-100.in	0.82	1.39	1.97	1.97

A one-thread run is *slower* than serial here — 0.81–0.94× — the fork/join and reduction overhead paid for no parallelism, which is why the one-thread column is the honest baseline to quote rather than assuming 1.0. Speedup peaks at four threads and falls back at eight on every instance, on a machine with eight performance cores.